

El lenguaje ensamblador del MIPS R2000

Ing. Walter Lozano
Ing. Alejandro Rodríguez Costello

Junio 2019

1. Introducción al manejo de memoria

El lenguaje ensamblador o **assembly** es la representación simbólica de la codificación binaria de un programa en **lenguaje máquina**. El lenguaje máquina está constituido por instrucciones máquina que indican al procesador lo que tiene que hacer, es decir, son las órdenes que el procesador es capaz de comprender. Cada instrucción máquina está constituida por un conjunto ordenado de unos y ceros, organizados en diferentes **campos**. Cada uno de estos campos contiene información que se complementa para indicar al procesador la acción a realizar.

El lenguaje ensamblador ofrece una representación más próxima al programador y simplifica la lectura y escritura de los programas. Las principales herramientas que proporciona la programación en ensamblador son: la utilización de nombres simbólicos para operaciones y posiciones de memoria, y determinados recursos de programación que permiten aumentar la claridad (legibilidad) de los programas. Entre estos recursos cabe destacar: la utilización de directivas, pseudoinstrucciones y macros que permiten al programador ampliar el lenguaje ensamblador básico definiendo nuevas operaciones.

Una herramienta denominada programa ensamblador o **compilador de assembly** traduce un **programa fuente** escrito de forma simbólica, es decir, en assembly, a instrucciones máquina. El programa ensamblador lee un fichero fuente escrito en lenguaje ensamblador y genera un fichero **objeto**. Este fichero contiene instrucciones en lenguaje máquina e información que permite combinarlo con otros de ser necesario. Otra herramienta denominada **linker**, es la encargada de combinar estos ficheros objeto para crear un único fichero llamado **ejecutable** que puede correr en la computadora.

En el simulador utilizado, para cargar un programa en memoria se utiliza la opción *load* o el editor de texto incorporado y luego se selecciona la opción *compile*, que lleva a cabo todos los pasos necesarios, desde la traducción del programa ensamblador a lenguaje máquina hasta la carga del mismo en memoria. En caso de encontrar errores sintácticos de ensamblado aparece un mensaje de error en la ventana de mensajes y, a partir de esa instrucción, no se traduce ni se carga en memoria ninguna otra.

Llamaremos identificador a cualquier secuencia de caracteres alfanuméricos, guiones bajos y puntos, que no comienzan con un número. Ahora definiremos algunos conceptos básicos, referidos a los recursos de programación en assembly que facilitan su uso:

- **Comentarios:** Estos son muy importantes en los lenguajes de bajo nivel ya que ayudan a seguir el desarrollo del programa y, por tanto, se usan con profusión. Comienzan con un carácter “#” y a partir del mismo el compilador ignora el resto hasta el final de la línea.
- **Códigos de operación:** Son palabras reservadas que no pueden ser utilizadas como identificadores y que representan una instrucción máquina del procesador. Son también llamados nemotécnicos o vulgarmente instrucciones en assembly aunque no sea estrictamente correcto. Debemos recordar que una instrucción se compone de su código y operadores de ser necesario.
- **Etiquetas:** Son identificadores que se sitúan al principio de una línea y seguidos de dos puntos. Sirven para hacer referencia a la posición o dirección de memoria del elemento definido en ésta. A lo largo del programa se puede hacer referencia a ellas en los **modos de direccionamiento** de las instrucciones.
- **Pseudoinstrucciones:** Son instrucciones que no tienen traducción directa al lenguaje máquina que entiende el procesador, pero el ensamblador las interpreta y las convierte en una o más instrucciones máquina reales. Permiten una programación más clara y comprensible. A lo largo del desarrollo de las prácticas se las irán introduciendo para facilitar la programación.
- **Directivas:** Tampoco son instrucciones que tienen traducción directa al lenguaje máquina que entiende el procesador. Podemos decir que son instrucciones para el ensamblador. Este la interpreta y le informa cómo debe traducir el programa. Son identificadores reservados, que el compilador reconoce y que van precedidos por un punto.

Adicionalmente señalaremos que los números se escriben, por defecto, en base 10. Si van precedidos de 0x, se interpretan en hexadecimal. Las cadenas de caracteres se encierran entre comillas dobles (“”). Los caracteres especiales en las cadenas siguen la convención del lenguaje de programación C.

El primer paso para el desarrollo de un programa en ensamblador es definir los datos en memoria. La programación en ensamblador permite utilizar directivas que facilitan reservar espacio de memoria para datos e inicializarlos a un valor. En esta sección veremos el uso de estas directivas.

En primer lugar, recordar que, aunque la unidad base de direccionamiento es el byte, la memoria del R2000 tiene un ancho de 4 bytes o 32 bits, que se llama palabra o **word**, el mismo ancho que el del bus de datos. Así pues, cualquier acceso a una palabra de memoria supondrá leer cuatro bytes (el byte con la dirección especificada y los tres almacenados en las siguientes posiciones). Las direcciones de palabra deben estar alineadas en posiciones múltiplos de cuatro. Otra posible unidad de acceso a memoria es transferir media palabra (half-word).

1.1. Declaración de palabras en memoria

La directiva `.data [dir]` indica que los elementos que se definen a continuación se almacenarán en la zona de datos indicada por la dirección `dir`. De no

aparecer ninguna dirección como argumento de dicha directiva, la dirección de almacenamiento será por defecto `0x10010000`. La directiva `.word num` reserva espacio para un entero de tamaño `word` (4 bytes).

Luego de cargar el siguiente código, localiza los datos inicializados en memoria en el panel *Data Segment* con su valor en hexadecimal. Reflexiona en qué direcciones se han almacenado y porqué. ¿Cuáles son los valores que toman las etiquetas `palabra1` y `palabra2`?

```
.data          # comienza zona de datos
palabra1: .word 15    # decimal
palabra2: .word 0x15  # hexadecimal
```

Ahora utiliza el siguiente código, comprueba si hay diferencias con el anterior y reflexiona.

```
.data 0x1001000C # comienza zona de datos
palabras: .word 15, 0x15    # decimal/hexadecimal
```

Escribe un código que defina un vector de cinco palabras que esté asociado a la etiqueta `vector`, que comience en la dirección `0x10000000` y sus los valores sean `0x10`, `30`, `0x34`, `0x20` y `60`. Comprueba que inicializa la memoria de forma correcta. ¿Qué ocurre si se quiere que el vector comience en la dirección `0x10000002`? ¿En qué dirección comienza realmente y por qué?

1.2. Declaración de bytes en memoria

La directiva `.byte valor` inicializa una posición de memoria de tamaño *byte*, con el contenido `valor`.

Carga el siguiente código y reflexiona en cuál dirección de memoria se almacenó el valor especificado.

```
.data
octeto: .byte 0x10
```

Ahora estudia como se comporta el siguiente programa examinando con mucha atención el segmento de datos:

```
.data
palabra1: .byte 0x10,0x20,0x30,0x40
palabra2: .word 0x10203040
```

¿A que se debe el cambio de organización entre `palabra1` y `palabra2`? Ayuda: Investiga en Internet sobre endianness¹.

1.3. Declaración de cadenas de caracteres

Las directivas `.ascii cadena` y `.asciiz cadena` permiten cargar en posiciones de memoria consecutivas de a un byte, el código ASCII de cada uno de los caracteres que componen el string `cadena`. La única diferencia entre ellas es que la primera no coloca un terminador nulo al final y la segunda si. Podemos comprobar su funcionamiento analizando como queda la memoria al cargar el primer código y luego reemplazar la directiva `.ascii` por `.asciiz`. Finalmente verifica cuál sería el código equivalente utilizando solo la directiva `.byte`.

¹<https://es.wikipedia.org/wiki/Endianness>

```

        .data
cadena: .ascii "abcde" # caracteres consecutivos
octeto: .byte 0xff

```

Nota: es importante entender que en assembly no existe la noción de string a nivel máquina y por lo tanto la directiva `.ascii/z` no define un string en el sentido en que se le asigna en C sino que solo ayuda a inicializar la memoria con o sin un terminador nulo.

1.4. Reserva de espacio en memoria y alineación

La directiva `.space n` sirve para reservar espacio en memoria inicializándola a valor 0, donde `n` es el número de bytes reservados. Es particularmente útil cuando se debe inicializar grandes regiones de memoria.

Carga el siguiente código y analiza cuál es el rango de posiciones de memoria que se ha reservado para la etiqueta `espacio`, cuantos bytes y cuantas palabras.

```

        .data
palabra1: .word 0x20
espacio: .space 8 # reservo espacio
palabra2: .word 0x30

```

Al utilizar la directiva `.space` en combinación con otras se corre el riesgo de hacer una reserva de direcciones que no esté alineada a nivel palabra. Por ejemplo para el siguiente código la dirección de `espacio` no corresponde a ninguna dirección válida de palabra. Para entender el porqué reflexione sobre el rango de direcciones que se ha reservado para `espacio` y las direcciones de `byte1`, `byte2` y `entero`. Si una instrucción orientada a palabras accede a la dirección de `espacio` se producirá un error y el procesador detendrá su ejecución.

```

        .data
byte1:   .byte 0x10
espacio: .space 4
byte2:   .byte 0x20
entero:  .word 10

```

La situación anterior puede solucionarse alineando al dirección de la reserva de `espacio`. Para ello se utiliza la directiva `.align x` donde `x` es 1, 2 o 4 e indica si queremos alinear a half word, word o double word respectivamente. Para entender su funcionamiento cargue el siguiente código y analice nuevamente todas las direcciones de las etiquetas mencionadas.

```

        .data
byte1:   .byte 0x10
        .align 2
espacio: .space 4
byte2:   .byte 0x20
entero:  .word 10

```

1.5. Problemas propuestos de la sección

1. Diseña un programa ensamblador que reserva espacio para dos vectores A y B de 20 palabras cada uno a partir de la dirección 0x10000000.

2. Diseña un programa ensamblador que realice la siguiente reserva de espacio en memoria a partir de la dirección `0x10001000`: una palabra, un byte y otra palabra alineada en una dirección múltiplo de 4.
3. Diseña un programa ensamblador que realice la siguiente reserva de espacio e inicialización en memoria a partir de la dirección por defecto: 3 palabras, un byte conteniendo `0x10`, luego 4 bytes a partir de una dirección múltiplo de 4 y 20 bytes.
4. Diseña un programa ensamblador que defina, en el espacio de datos, la siguiente cadena de caracteres: `"Esto es un problema"` utilizando las directivas a) `.ascii`, b) `.byte`, c) `.word`.
5. Sabiendo que un entero se almacena en un word, diseña un programa ensamblador que defina en la memoria de datos, la siguiente matriz A de enteros a partir de la dirección `0x10010000` suponiendo que: a) la matriz A se almacena por filas y b) la matriz A se almacena por columnas.

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$